

Technology Focus areas

Java Springboot

Java Concepts Overview

Core Java Fundamentals	✓ JVM, JDK, JRE — architecture & lifecycle ✓ Compilation: <code>javac</code> → bytecode → JVM ✓ Memory model: Stack vs Heap ✓ Key concepts: <code>final</code>/<code>finally</code>/<code>finalize</code>, <code>static</code>, <code>volatile</code>, String immutability
Object-Oriented Programming (OOP)	✓ 4 Pillars: Encapsulation, Abstraction, Inheritance, Polymorphism ✓ Overloading vs Overriding, Abstract vs Interface ✓ Multiple inheritance via default methods, DTO usage
Collections Framework	✓ List, Set, Map — internal logic & use cases ✓ ArrayList vs LinkedList, HashMap internals ✓ <code>hashCode()</code> & <code>equals()</code>, ConcurrentHashMap, Generics
Java 8+ Features	✓ Lambdas, Functional Interfaces, Streams API ✓ Optional class, Method References ✓ Predicate, Function, Consumer — key interfaces
Modern Java (11–21)	✓ Sealed classes, Records, Text blocks ✓ <code>var</code> keyword, Pattern Matching, Switch enhancements
Exception Handling	✓ Checked vs Unchecked, Multi-catch, Custom exceptions ✓ Common errors (NPE, OOM), Best practices

Java Concepts Overview cont..

Multithreading & Concurrency	✓ Thread lifecycle, Runnable/Callable/Executor ✓ Synchronization, volatile, Concurrent utilities ✓ CompletableFuture essentials
Serialization	✓ Serializable, transient, readResolve() for Singletons
Spring Boot Concepts	✓ @Controller vs @RestController, @Autowired, @Transactional ✓ Transaction management, AOP, JPA integration, Logging
Coding & Problem Solving	✓ Reverse string, find duplicates, 2nd/3rd highest values ✓ HashMap frequency, Stream transformations ✓ Palindrome & sorting with Comparator

Spring boot concepts overview

Why Spring Boot?	✓ Simplifies classic Spring — auto-configuration, embedded servers, minimal XML. ✓ Ideal for microservices — faster startup, lighter deployment. ✓ Spring Boot ≠ Spring Framework — Boot adds automation & opinionated defaults. ✓ Key features: @SpringBootApplication, Auto-config, application.yml, Profiles (spring.profiles.active=dev), IoC Container.
Core Spring Annotations	✓ @Controller, @RestController, @Service, @Repository, @Component — Stereotypes ✓ DI: @Autowired, @Qualifier, @Primary, @Value ✓ Config: @Bean, @Configuration, @ComponentScan ✓ Advanced: @EnableConfigurationProperties, @ConfigurationProperties ✓ Custom annotations & Bean scopes (Singleton, Prototype)
Dependency Injection / IoC	✓ IoC manages object creation & wiring automatically. ✓ @Autowired vs new object creation. ✓ Handle conflicts: @Primary, @Qualifier. ✓ Common errors: Unsatisfied dependencies, duplicate beans.
JPA & Hibernate Integration	✓ ORM Layer → JPA (standard), Hibernate (implementation). ✓ Annotations: @Entity, @Table, @Id, @Column. ✓ CRUD via JpaRepository / CrudRepository. ✓ Queries: JPQL / Native, Pagination & Sorting.

Spring boot concepts overview cont..

Database Config	✓ application.properties: DB URL, username, dialect. ✓ Driver config & connection pooling (HikariCP). ✓ XML vs Annotation-based configs.
REST API Layer	✓ Methods: GET, POST, PUT, DELETE. ✓ Mappings: @GetMapping, @PostMapping... ✓ Params: @RequestBody, @PathVariable, @RequestParam. ✓ ResponseEntity, HTTP codes, Async (@Async).
Exception Handling & AOP	✓ Global error handling: @ControllerAdvice, @ExceptionHandler. ✓ AOP: Aspects, Join Points, Advice types for cross-cutting logic.
Transactions & Persistence	✓ @Transactional → Propagation, Isolation, Rollback. ✓ Cascade, Lazy/Eager loading. ✓ Data consistency in concurrent environments.
Microservices Concepts	✓ Monolith → Microservices = scalability & isolation. ✓ Communication: REST, Feign Client. ✓ Service Discovery: Eureka.
Advanced Topics	✓ Profiles (dev/test/prod), Actuator, Async/Scheduled tasks. ✓ Performance tuning (HikariCP), handling BLOB/CLOB.
Coding & Problem Solving	✓ CRUD using JpaRepository. ✓ Native query with sort/pagination. ✓ Async API & microservice call example.

Java & Spring Boot Coding Assessment

Concept	TOPIC	FOCUS AREA
String Manipulation & Character Ops	Work with String, StringBuilder, and Character APIs — anagrams, reversals, duplicates, frequency counts.	• Immutability • HashMap usage • Streams (groupingBy, counting) • Character iteration
Arrays & Collections Logic	Algorithmic logic: frequency counts, Two-Sum, missing numbers, deduplication, Kadane's algo.	• Iteration, HashMap/HashSet • Streams (filter, map, distinct) • Sorting/searching • Optimization
Linked List & DS Ops	Custom DS creation: create/reverse list, swap strings, immutability logic.	• Node structure, recursion vs iteration • Object references • Final & immutability
Java 8 & Streams API	Functional style filtering & transformations (map, filter, collect).	• Lambdas, Functional Interfaces • Comparators • Stream pipeline (map → filter → collect)
OOP & Collections	Class design, equality, sorting employees, handling duplicates.	• Encapsulation • equals()/hashCode() • Comparator/Comparable • Custom object keys
Exception Handling	Robust code: handle & define exceptions.	• Checked vs Unchecked • Custom exceptions • try-catch-finally • Multi-catch
Logical & Mathematical Programming	Loops, recursion, and numeric pattern logic.	• Fibonacci, sum logic, arrangement patterns • Loop optimization • Conditional reasoning
Sorting & Lambdas	Custom sorting using streams and comparators.	• Comparator.comparing() • Method references • Natural vs custom order
Pattern & Problem Solving	Output patterns, seating or arrangement algorithms.	• Algorithmic reasoning • Nested loops • Data ordering logic
Conceptual Checkpoints	Theory & behaviour validation.	• == vs equals() • String pool • HashMap internals • Memory & debugging awareness